

Architecting an *Institutional-Grade* Double-Entry Banking Ledger

Why eventual consistency fails for money, and how we engineered mathematical correctness, deterministic deadlock-free locking, and cryptographic auditability in Go & PostgreSQL.

GO (GOLANG)

POSTGRESQL

ACID TRANSACTIONS

DEADLOCK PREVENTION

SHA-256 AUDIT TRAIL

Why Naive Financial Ledgers *Fail Catastrophically*

01

IEEE-754 Float Traps

Using `float64` causes binary rounding errors ($0.1 + 0.2 = 0.30000000000000004$). Over millions of transactions, financial balance drifts and money vanishes into thin air.

02

Deadlock Race Conditions

When User A transfers to B at the exact same millisecond that B transfers to A, random row lock acquisition causes database deadlocks and aborted transactions under high concurrency.

03

Network Replay Double-Spend

Mobile apps and payment gateways retry timed-out requests. Without SHA-256 payload-aware idempotency, network retries double-debit customer accounts.

04

Retroactive DB Tampering

Standard mutable databases allow rogue queries to alter historical transaction amounts without leaving a mathematical or cryptographic signature.

Strict Double-Entry: $\Sigma Debits \equiv \Sigma Credits$

The Universal Law of Value Conservation

Money cannot be created or destroyed. In ApexLedger, every single transaction strictly satisfies:

$$\Sigma \text{ Debit Legs} - \Sigma \text{ Credit Legs} \equiv 0$$

Deposits and withdrawals do not create value from thin air; they transact against a designated counterparty: **System CASH**(000000000-0000-0000-0000-0000000000001).

Dual Safety Net

Enforced at two independent layers: (1) Go Service Business Logic, and (2) PostgreSQL database **CHECK** (`type = 'system' OR balance >= 0`) constraint.

```
// 1. Validate Double-Entry Balance
var totalDebit, totalCredit int64
for _, leg := range legs {
    if leg.Direction == domain.DirectionDebit {
        totalDebit += leg.Amount
    } else {
        totalCredit += leg.Amount
    }
}
if totalDebit != totalCredit {
    return domain.ErrDoubleEntryImbalance // Strict Reject
}

// 2. Deposit Execution (Counterparty CASH)
legs := []domain.Leg{
    {AccountID: CASH_VAULT, Direction: "debit", Amount: amt},
    {AccountID: customerID, Direction: "credit", Amount: amt},
}
```

Minor-Unit Integer Arithmetic: *Never Use Float64*

The Dangerous IEEE-754 Floating-Point Flaw

In binary floating-point representation, base-10 fractions (like 0.10 or 0.70) cannot be represented cleanly. Over recurring interest calculations, tax deductions, or batch splits, microscopic rounding errors accumulate into massive financial discrepancies.

The ApexLedger Standard: Signed Int64 Minor Units

All currency in ApexLedger is represented strictly in atomic minor units:

- **Indian Rupee (INR):** Stored as *Paise*(× 100)
- **US Dollar (USD):** Stored as *Cents*(× 100)
- **Conversion Rule:** Division by 100 happens strictly at the visual UI / API output boundary.

```
// PostgreSQL Schema: BIGINT Minor Units
CREATE TABLE accounts (
  id          UUID PRIMARY KEY,
  owner       TEXT NOT NULL,
  type        TEXT NOT NULL CHECK (type IN ('customer','system')),
  currency    CHAR(3) NOT NULL DEFAULT 'INR',
  balance     BIGINT NOT NULL DEFAULT 0, -- Int64 Paise
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT no_negative CHECK (type = 'system' OR balance ≥ 0)
);

CREATE TABLE entries (
  id          BIGSERIAL PRIMARY KEY,
  transaction_id UUID NOT NULL REFERENCES transactions(id),
  account_id  UUID NOT NULL REFERENCES accounts(id),
  direction   TEXT NOT NULL CHECK (direction IN ('debit','credit')),
  amount      BIGINT NOT NULL CHECK (amount > 0), -- Strict Positive
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Deterministic Row-Locking: *Eliminating Deadlocks*

The Classic Circular Wait Trap

If Goroutine 1 locks Account A and waits for B, while Goroutine 2 locks B and waits for A, PostgreSQL encounters an unresolved dependency cycle and forcibly aborts with a Deadlock Error.

ApexLedger's Deterministic Total Order Lock

Before acquiring row locks, ApexLedger sorts all participating account UUIDs in strict lexicographical order:

`min(UUID_A, UUID_B) → max(UUID_A, UUID_B)`

Because every concurrent transaction acquires locks in the exact same sequence, **circular wait conditions are mathematically impossible**.

```
// 1. Sort Account IDs deterministically to prevent deadlocks
ids := []uuid.UUID{from, to}
sort.Slice(ids, func(i, j int) bool {
    return ids[i].String() < ids[j].String()
})

// 2. Acquire Pessimistic Row Locks in strict order
rows, err := tx.Query(ctx,
    SELECT id, balance
    FROM accounts
    WHERE id = ANY(
        ORDER BY id FOR UPDATE, ids)
if err != nil {
    return nil, err
}
defer rows.Close()
```

Cryptographic Hash Chaining: *Tamper-Proof Audit Trail*

Mathematical Provability Over Blind Trust

Every journal entry is cryptographically bound to its parent entry using a SHA-256 hash chain:

```
entry_hash = SHA256(prev_hash + account_id + direction + amount)
```

If an attacker with database access alters historical amounts or inserts ghost records, the hash chain immediately breaks, triggering an automated reconciliation alarm.

Automated Continuous Auditing

The platform exposes an automated audit verification endpoint (`GET /admin/verify-audit-chain`) that recalculates and validates the entire cryptographic chain on demand.

```
// Cryptographic SHA-256 Hash Chaining Function
func computeEntryHash(prevHash string, accID uuid.UUID, dir string, amt int64) string {
    record := fmt.Sprintf("%s:%s:%s:%d", prevHash, accID.String(), dir, amt)
    sum := sha256.Sum256([]byte(record))
    return hex.EncodeToString(sum[:])
}

// Audit Chain Verification
func (s *Store) VerifyAuditChain(ctx context.Context) (*Report, error) {
    // Traverses all entries in order
    // Recomputes expected SHA-256 hashes
    // Flags any broken cryptographic link
}
```

Payload-Hashed Idempotency: *Safe Retries & Zero Double-Debits*

The Blind UUID Flaw

Most APIs only store an idempotency key string. If a malicious or buggy client sends the same key with a different amount or different recipient, naive systems replay the wrong data or cause state corruption.

The ApexLedger Solution: Payload Hashing

ApexLedger stores the key alongside the **SHA-256 hash of the complete HTTP request body**:

- **Exact Replay**: Same key + Same body → Replays original response with **Idempotent-Replay: true**.
- **Key Poisoning Rejection**: Same key + Different body → Rejected with **422 Unprocessable Entity**.
- **Concurrent Race**: Second simultaneous request blocks on unique row lock, preventing race duplicates.

```
// Atomically claim idempotency key
tag, err := tx.Exec(ctx,
    INSERT INTO idempotency_keys (key, request_hash)
    VALUES (, )
    ON CONFLICT (key) DO NOTHING, key, reqHash)

if tag.RowsAffected() == 0 {
    // Key already claimed: Check stored hash
    if storedHash != reqHash {
        return nil, domain.ErrKeyReused // 422 Conflict
    }
    if status == nil {
        return nil, domain.ErrInProgress // 409 In Flight
    }
    return &Result{Replayed: true, Status: *status, Body: body}, nil
}
```

Two-Phase Holds & *Multi-Party Split Settlements*

01

Two-Phase Commit Pre-Auth Holds

In card processing and hospitality, funds must be reserved without moving settled ledger capital:

$$\text{Available Balance} = \text{Total Balance} - \text{Active Holds}$$

- **Create Hold:** Ring-fences available balance immediately.
- **Capture Hold:** Turns hold into settled debit transaction.
- **Void Hold:** Cancels reservation; frees ring-fenced liquidity.

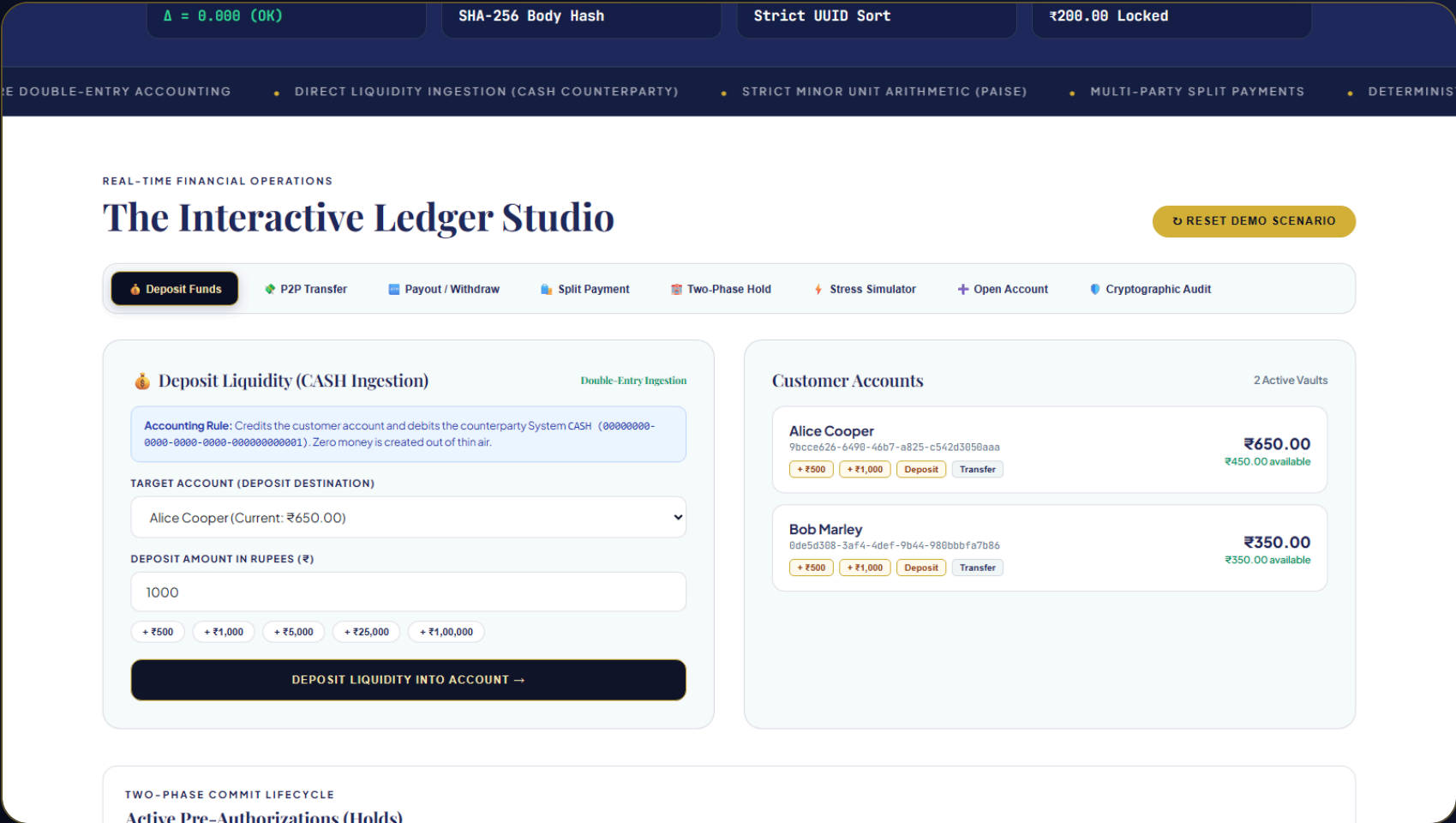
02

Atomic Multi-Party Split Settlements

Modern marketplace checkouts require 1 buyer payment to disburse across multiple vendors atomically:

- **Buyer Account:** Debited -â,11,000.00
 - **Merchant Account:** Credited +â,1950.00 (95% Net Revenue)
 - **Platform Fee Escrow:** Credited +â,150.00 (5% Platform Fee)
- ⇒ **Net Transaction Imbalance ≡ 0.00**

Interactive Studio & *High-Concurrency Race Simulator*



Live Stress Simulator Results

- **10 Simultaneous Goroutines:** Alternating cross-transfers between Alice and Bob.
- **Zero Deadlocks:** Deterministic row lock order eliminates contention.
- **Zero Drift:** Automated global reconciliation proves debits strictly equal credits.
- **Visual T-Account Inspector:** Click any journal entry to inspect balanced legs, SHA-256 hashes, and trigger 1-click reversals.

Full Observability Stack

Prometheus metrics exposed at `/metrics` tracking transaction duration histograms, total TPS counters, active holds, and structured logs via Go's `log/slog`.

Production Architecture & *Key Takeaways*

Mathematical Guarantees

Financial software cannot rely on eventual consistency. Using strict double-entry and minor-unit int64 arithmetic ensures 100% balance preservation.

Deterministic Concurrency

Sorting account UUIDs before issuing **FOR UPDATE** locks eliminates circular wait deadlocks under high multi-threaded traffic.

Immutable Cryptographic Seal

Append-only journal entries with SHA-256 block hash chaining turn the database into an immutable, tamper-evident audit record.

Explore the Full Open-Source Repository

<https://github.com/silentguyracer/Mini-banking-ledger-API->

Engineered by **Sahil Kumar** • Connect with me on LinkedIn for distributed systems & backend architecture discussions!